TD 2 - The quorum, then the routing rule

Beta · 2026-10-18-students-1, 18 October 2026

Beta: this resource may still change before its session. The site is updated in place; the pack version above identifies what you read.

> Beta. Pack 2026-10-18-students-1. Set your machine up with Before TD 1.

Module: Trustworthy generative AI. Session 2 (18 November), 13:30 to 18:00.

In TD 1, you measured five models separately and placed each one in the portfolio. This session builds the two decisions of a service: having several models vote when the answer carries consequences, and assigning each question to the cheapest model that remains up to it. By 18:00, your vote runs on the twenty questions, you have explained a wrong-consensus case from your batch or a supplied example, and your routing rule is written and exercised on a simulator.

What you can do by the end of the session

- Implement a k-of-n vote on a panel of distinct families, with a strict majority, which keeps a value or abstains without manufacturing an answer.
- Identify a wrong consensus in traces, explain its cause and draw the consequence for the choice of panel and of checking.
- Write a routing rule per question family, justified by the TD 1 results, which provides for the quorum, the deferral and the abstention.
- Read the decisions of a resource simulator and adapt its policy to your rule.

Material provided

- Your TD 1 traces (`traces-td1.csv`, merged between the members) and your models x questions table with the portfolio sheet.
- `skeleton-multi-llm-en.py`: the `Orchestrateur.vote_k_sur_n` method is to be completed; the rest of the skeleton does not change.
- `atelier-vote.py`: exercises your vote on four synthetic examples, then on the real traces question by question, with a fixed panel of three families: `openai/gpt-oss-120b` (Groq), `cohere/north-mini-code:free` (OpenRouter), `qwen/qwen3.8-27b` (Groq).
- `atelier-routage.py`: a routing simulator on three fictitious resources and seven scenarios, without any call to a provider.
- portefeuille-modeles-en and its CSV, for the prices and the access routes.
- Your stored TD 1 traces. An optional new experiment uses a separate allocation agreed with the instructor and current account allowances.

Schedule

Time	Step	Expected output
13:30 to 13:45	Roles, state of the TD 1 traces	merged traces, one table per sub-group
13:45 to 15:00	**Step 1**: the k-of-n vote in the skeleton, exercised on the traces	`vote_k_sur_n`, decisions on twenty questions
15:00 to 15:45	**Step 2**: at least one wrong consensus, its cause, the consequence	commented case
15:45 to 16:00	Break	
16:00 to 17:15	**Step 3**: the routing rule, written then exercised on the simulator	one-page rule, simulator outputs
17:15 to 17:45	**Step 4**: plenary	collective finding
17:45 to 18:00	Marking of the session	Q1, Q2 and Q3 marked on the sheet

Step 1 - The k-of-n vote (13:45 to 15:00)

Complete `Orchestrateur.vote_k_sur_n(self, traces_question, k)` in `skeleton-multi-llm-en.py`. Each element of `traces_question` carries `provider`, `model_requested`, `value` (an integer in cents, or `None`) and `verdict` (`executed` or `error`). The contract:

1. only the executed proposals with a value vote; an error or an unavailability stays in n without a vote;
2. k is a strict majority: `n // 2 < k <= n`, otherwise the method refuses;
3. two proposals from the same provider and model pair are refused: a duplicate is not a second vote;
4. if a value reaches k votes, the method returns `{"consensus": True, "value": v, "votes": nombre, "models": [...]}`;
5. otherwise it returns `{"consensus": False, "value": None, "votes": plus grand nombre de voix, "models": []}`. It never invents a value.

Exercise the vote:

```
python3 atelier-vote.py                                    # four synthetic
examples, no network
python3 atelier-vote.py --traces traces-td1.csv --case C02 # the real traces of one
question
```

The second call takes, for each member of the panel, its last attempt on the question, decides, then compares the value kept with your reference. The comparison comes after the decision: the vote does not know the reference. Run the command again for the twenty questions and record, per question: consensus or abstention, value, votes, agreement with the reference.

Use TD 1 traces for the required work: no new network batch is needed. Any optional repeat uses a separately agreed allocation, current account counters and distinct experiment files.

Deliverable of the step: the completed method, and the table of the twenty decisions of the vote.

Step 2 - The wrong consensus (15:00 to 15:45)

Find an identified case where at least two models return the same wrong value. Show the question, values, independent reference and cause in the SQL. State its nature: real batch, supplied historical real case or synthetic example. If your own batch contains none, use the instructor's case or the synthetic fallback supplied by `atelier-vote.py`; access to Q2 level 2 is unchanged.

Declare the panel and threshold k out of n, then show what that quorum returns. Agreement between two of the five TD 1 models does not establish the result of the fixed 2-of-3 panel. Dots is not a member of that panel. Explain an independent check that detects the shared error; a different panel or escalation is a candidate check, not a guarantee.

Deliverable: one explained case with its identifier, source/nature, agreeing models, reference, declared quorum result and checking method. Keep a supplied or synthetic case separate from your batch's measured counts.

Supplied synthetic case: DEMO-VOTE

The workshop's `wrong consensus` example has proposals `[0, 0, 100]`, reference 100 cents and threshold 2 of 3. To explain a possible shared SQL cause, use this invented sale: date `2026-01-01`, amount 100 cents. The requested period is 1 January inclusive to 2 January exclusive. Two proposals use `sale_date > '2026-01-01'`; the third uses `sale_date >= '2026-01-01'`. All use `sale_date < '2026-01-02'` and `COALESCE(SUM(amount_cents), 0)`. Execute both variants on this one-row toy table, explain the boundary and compare the quorum with the independent reference. These values and queries are constructed teaching material, not model outputs or answers on the shop database.

Step 3 - The routing rule (16:00 to 17:15)

Write, on one page, the rule of the sub-group. For each question family (period, completed age at a date, age at the order, product list, combinations):

- the first model called: the cheapest one, according to the portfolio, among those that your TD 1 table shows to be correct on the family;
- the checking: none, 2-of-3 quorum, 3-of-5 quorum, reference, review, according to what an error costs for three uses (internal dashboard, figure shown to the customer, amount paid out);
- the course of action if the first model is unavailable: next model, waiting for the refill when the request can wait, abstention.

Each line cites the cells of the TD 1 table that justify it. Keep simple period and product questions: they help compare routing choices. Twenty questions support a batch-specific hypothesis, not a general error rate, a permanent ranking or a quorum error guarantee.

Then launch the simulator:

```
python3 atelier-routage.py                       # the seven scenarios
python3 atelier-routage.py --scenario deferrable # one scenario
```

Read each decision (`run`, `defer`, `abstain`), its resource and its reason. Then adapt `RESOURCES` and `choose` to your rule: add your models with their level and their price, and write the reason that the trace must carry. The levels, availabilities and prices of the simulator are fictitious; those of your rule come from the portfolio and from TD 1.

Deliverable of the step: the one-page rule and the simulator outputs, before and after adaptation.

Step 4 - Plenary (17:15 to 17:45)

Each sub-group presents its wrong consensus and its rule. Compare the rules: the same database, the same traces, rules that differ by the checking retained and by the cost accepted. Note the questions for which no free model is enough on its own: they set the price of checking in TD 3.

The instructor then marks the session at each table: Q1, Q2 and Q3, each 0, 1 or 2, on the sheet laid on your table since TD 1.

What is expected (the evaluated artefacts)

1. `skeleton-multi-llm-en.py` with `vote_k_sur_n` completed.
2. The table of the twenty decisions of the vote: consensus or abstention, value, votes, agreement with the reference.
3. At least one commented wrong consensus: question, value, models, cause, consequence.
4. The one-page routing rule, with the TD 1 cells that justify it.
5. The simulator outputs, before and after adaptation.
6. The prompts, as used, if they have changed since TD 1.

Hand-in and session report

At the end of TD 2, add `reports/td2.md` to your sub-group's public Git repository, shared from TD 1 to TD 3. Describe the work attempted and completed, successes and failures with links to the results and traces, observed causes and hypotheses, limitations, each member's contribution and next steps. Identify replayed or synthetic examples explicitly.

Commit the report with the code, prompts and evidence. Update your entry in the same Moodle hand-in area with the full repository URL, the session's full commit hash and a link to the report at that commit. Retain earlier TD entries and reports. Check that the repository opens without signing in. Follow the hand-in dates announced in Moodle.

What is valued

- **A vote that abstains** when no value reaches k, and that keeps an unavailable member in n.
- **A wrong consensus explained** by the returned queries, with the consequence for the panel or the checking.
- **A rule justified by the traces**: each choice of model refers back to cells of TD 1, each check to a cost of error.
- **A readable reason** in each decision of the simulator.